



Projet – illico-mande – Rapport



Année universitaire : 2023 – 2024

Matière : Modélisation des systèmes informatiques

Professeur encadrant : M. John CHAUSSARD

Réalisé par : Hamza LAOUNI & Naoufal BENAMAR

Engagement de non-plagiat

Nous, soussignés Naoufal BENAMAR et Hamza LAOUNI, étudiants en première année d'école d'ingénieur à Sup Galilée,

Déclarons être pleinement conscients que la copie de tout ou partie d'un document, quel qu'il soit, publié sur tout support existant, y compris sur Internet, constitue une violation du droit d'auteur ainsi qu'une fraude caractérisée, tout comme l'utilisation d'outils d'Intelligence Artificielle pour générer une partie de ce rapport ou du code associé.

En conséquence, nous déclarons que ce travail ne comporte aucun plagiat, et assurons avoir cité explicitement, à chaque fois que nous en avons fait usage, toutes les sources utilisées pour le rédiger.

Fait à Villetaneuse, le 28/06/2024

Signatures :



Table des matières

I. Introduction	6
II. Définitions et outils utilisés	7
III. Diagramme de cas d'utilisation du système	8
IV. Diagramme de classe du système	17
V. Diagramme d'états-transition d'une Commande	24
VI. Conclusion	26

Table des figures

Figure 1 : Partie du diagramme de cas d'utilisation (côté Client)	9
Figure 2 : Diagramme de cas d'utilisation (côté Client)	11
Figure 3 : Diagramme de cas d'utilisation (côté Serveur)	12
Figure 4 : Diagramme de cas d'utilisation (côté Cuisinier)	13
Figure 5 : Diagramme de cas d'utilisation (côté Administrateur)	15
Figure 6 : Diagramme de cas d'utilisation réduit	16
Figure 7 : Diagramme de classe (côté Application)	17
Figure 8 : Lien entre Table, Commande et Serveur, Cuisinier	19
Figure 9 : Interactions que peut avoir le Client avec Menu, Panier et Commande	20
Figure 10 : Diagramme de classe complet	22
Figure 11 : Diagramme d'états-transition d'une Commande	25

1. Introduction

Dans le cadre de notre formation en première année cycle d'ingénieur, nous avons eu l'opportunité de mener à bien un projet de fin de matière "Modélisation des systèmes informatiques" en binôme. Notre binôme est composé de Naoufal BENAMAR et Hamza LAOUNI. Ce projet a été proposé par M. John CHAUSSARD, responsable de la matière et de la première année en Informatique à l'Ecole d'ingénieurs Sup Galilée.

Ce rapport a pour vocation d'exposer clairement le déroulement de notre projet, autrement dit, les différentes étapes par lesquelles nous sommes passés, les contraintes que nous avons rencontrées et les éventuelles améliorations que nous avons apporté à nos diagrammes au fur et à mesure de notre avancement.

L'objectif de ce projet est de modéliser un système de commande pour un restaurant, appelé Illico-mande, qui permet aux clients de passer et payer leurs commandes sans l'intervention d'un employé. Le processus est simple : le client, en s'asseyant à une table, peut scanner un QR Code qui le redirige vers une page web spécifique à sa table. Sur cette page, le menu du restaurant est affiché et le client peut choisir ses produits, passer sa commande et effectuer le paiement immédiatement.

Notre système devra inclure diverses fonctionnalités essentielles telles que la création et la gestion de comptes clients, la consultation et la sélection des produits du menu, la gestion du panier d'achat, et le paiement via plusieurs méthodes sécurisées. En outre, le client doit pouvoir appeler un serveur pour des besoins spécifiques via la même interface. Du côté du personnel du restaurant, une application dédiée permettra aux administrateurs, cuisiniers et serveurs de gérer les commandes et les interactions avec les clients de manière efficace et organisée.

Pour modéliser ces contraintes, nous allons, dans un premier temps réaliser le diagramme de cas d'utilisation de notre système. Puis nous esquisserons le diagramme de classe de ce dernier. Enfin, nous élaborerons le diagramme d'états transition de la classe « commande ».

II. Définitions et outils utilisés

UML (Unified Modeling Language) est un langage de modélisation graphique que nous utiliserons pour concevoir notre système de commande de restaurant "illico-mande".

Dans notre projet, nous nous concentrerons sur trois types de diagrammes UML essentiels :

- Diagramme de cas d'utilisation : Il nous permettra de représenter les interactions entre notre système de commande et ses différents acteurs (clients, serveurs, cuisiniers, administrateurs). Ce diagramme montrera les principales fonctionnalités que le système doit accomplir comme la connexion via QR code, la passation de commande, le paiement, et la gestion des commandes côté personnel.
- Diagramme de classes : Il illustrera la structure de notre système, montrant les relations entre les différentes classes telles que Client, Commande, Menu, Produit, ainsi que les classes liées au personnel (Serveur, Cuisinier, Administrateur). Ce diagramme nous aidera à définir l'architecture du logiciel, incluant les attributs et méthodes de chaque classe, ainsi que leurs relations.
- Diagramme d'état-transition : Nous l'utiliserons spécifiquement pour modéliser le cycle de vie d'une commande dans notre système. Il montrera les différents états d'une commande (par exemple : passée, en préparation, prête à être servie, servie) et les transitions entre ces états.

Chaque diagramme est conçu pour répondre à un besoin spécifique de modélisation, mais ensemble, ils fournissent une vue complète et cohérente du système. L'utilisation d'UML peut offrir plusieurs avantages dans le développement de logiciels. Tout d'abord, il peut aider à communiquer efficacement entre les membres de l'équipe de développement, en fournissant une base commune pour discuter et comprendre les spécifications du système. Il peut également aider à réduire les erreurs de conception, en fournissant une vue visuelle du système qui peut aider à identifier les problèmes potentiels. En résumé, UML est un langage de modélisation graphique utilisé pour concevoir, visualiser et documenter les systèmes logiciels à travers différents types de diagrammes.

Afin de réaliser nos diagrammes, nous utiliserons draw.io, un outil de diagramme en ligne gratuit. Il propose une interface simple, une large gamme de symboles UML, et permet la collaboration facile. Ses fonctionnalités d'exportation et sa compatibilité avec d'autres plateformes en font un choix pratique et efficace pour notre travail de modélisation.

III. Diagramme de cas d'utilisation du système

Nous allons ici identifier les différents acteurs, ainsi que les cas d'utilisation qui leurs sont associés.

Pour décrire précisément le diagramme de cas d'utilisation, nous dévoilerons progressivement les différentes parties du diagramme. Nous clarifierons également les limites de notre système ainsi que les hypothèses sur lesquelles nous nous sommes basés pour modéliser ce système.

Notre système est structuré autour de deux éléments principaux, complétés par un troisième : le QR code. Les deux composants majeurs sont une application dédiée au personnel et une page web conçue spécifiquement pour les clients.

1. Le Client :

- Scanner le QR Code : Le client utilise son smartphone pour scanner le QR code sur sa table, accédant ainsi à la page web spécifique à sa table (Session). Le client peut, ensuite, éventuellement, se connecter ou créer son propre compte.
- Créer un compte : Le client renseigne ses informations personnelles pour créer un nouveau compte dans le système (facultatif).
- Se connecter : Le client entre ses identifiants pour accéder à son compte existant (facultatif). Après s'être connecté, le client peut accéder à l'historique de ses commandes, ainsi qu'à sa cagnotte personnelle.

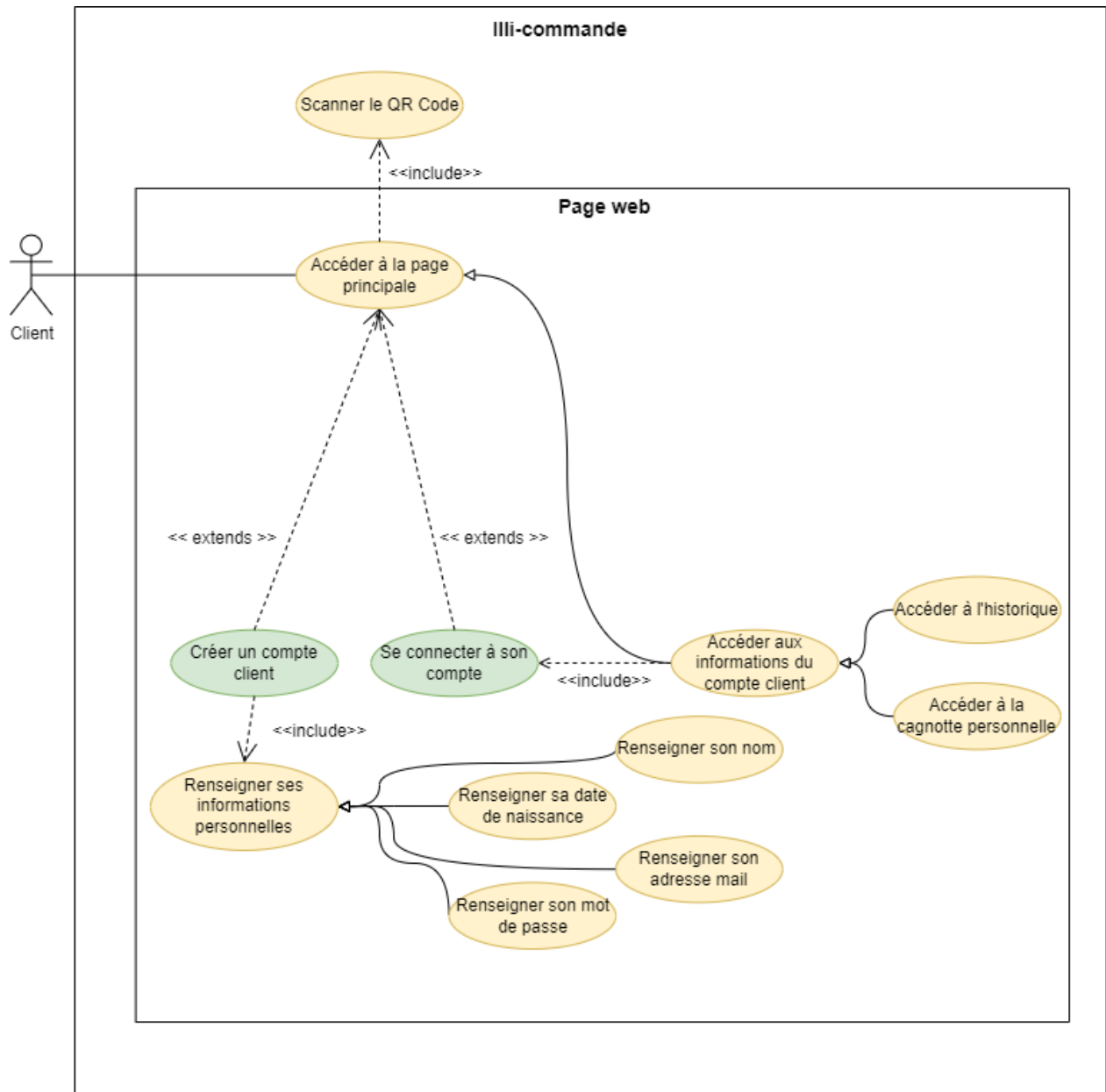


Figure 1 : Partie du diagramme de cas d'utilisation (côté Client)

Passons maintenant aux autres cas de notre diagramme.

Après avoir accédé à la page principale (connecté ou pas) à l'aide du QR Code, le client peut :

- Consulter le menu : Le client peut voir la liste complète des produits disponibles à la carte et voir les compositions des différentes formules (repas), les ingrédients des plats ainsi que le prix de chaque élément du menu.

- Ajouter des produits au panier : Après avoir judicieusement fait son choix ; le client sélectionne les produits qu'il souhaite commander et les ajoute à son panier virtuel.
- Consulter le panier : Le client peut retirer des éléments de son panier ainsi que le valider. Avant de passer au paiement, le client peut éventuellement laisser un pourboire.

N-B : On peut imaginer une conception où, au moment de valider le panier, un pop-up demande au client s'il souhaite laisser un pourboire.

- Payer la commande : Le client effectue le paiement de sa commande via la méthode choisie. Cette méthode peut être l'une des suivantes : Apple Pay, Google Pay, PayPal, carte de crédit, espèces.

N-B : Lorsque le client choisit la méthode de paiement par espèce ; le système (backend) alerte le serveur afin qu'il vienne encaisser le paiement.

De plus, lorsque le client choisit le paiement par carte de crédit, le client est redirigé vers une page de paiement sécurisée, gérée par une API de la banque.

Ici, on introduit, le système interbancaire, qui est considéré comme un acteur secondaire. Ce système gère tout ce qui est en relation avec le paiement d'une commande.

A noter que, c'est le système interbancaire qui est en relation directe avec le système « illico-mande » et non la banque.

- Sonner au serveur : Le client peut demander l'assistance d'un serveur en appuyant sur un bouton dans l'application. Après avoir fait cela, il peut demander une fourchette, de l'eau, un couteau, des serviettes supplémentaires ou bien poser une question spécifique à propos d'un plat.

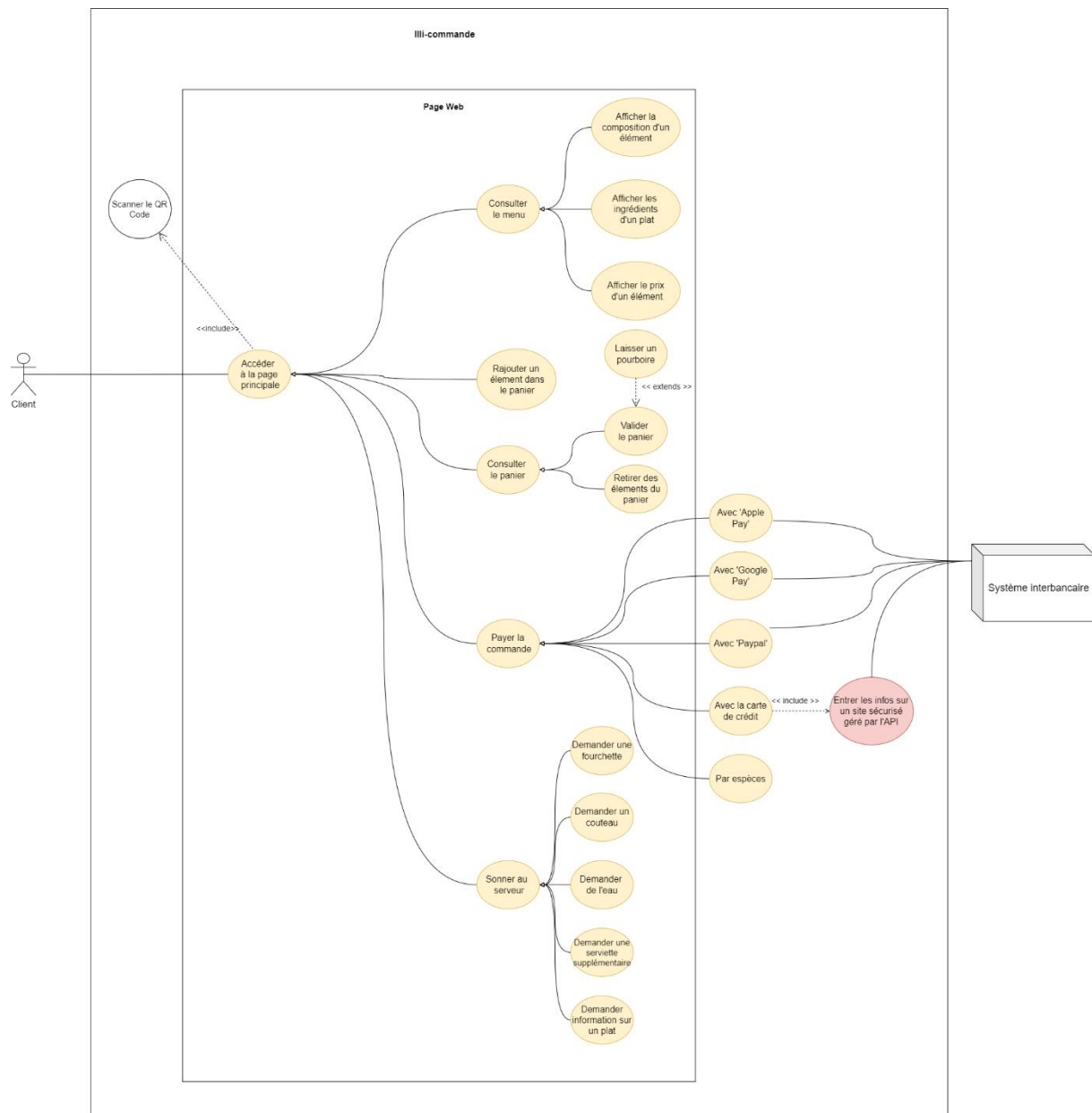


Figure 2 : Diagramme de cas d'utilisation (côté Client)

N-B : Nous sommes conscients que le schéma ci-dessus paraît illisible, MAIS (normalement) en zoomant, vous pourrez lire chaque cas d'utilisation sans aucun souci.

2. Le Serveur :

- Se connecter à l'application du personnel en tant que serveur : Le serveur accède à l'interface dédiée au personnel avec ses identifiants.
- Consulter les commandes assignées : Le serveur voit la liste des commandes pour les tables dont il est responsable et leur statut.
- Marquer les plats comme servis : Le serveur indique dans le système quand un plat a été servi au client. Il modifie donc un plat qui est « prêt à être servi » en « servi ».
- Recevoir des alertes (notifications) :

Le serveur est notifié :

- Par le système lorsqu'un cuisinier marque un plat comme « prêt à être servi ».
- Par le système lorsqu'un client demande de l'assistance (lorsqu'il sonne).
- Par le système lorsqu'un client choisit la méthode de paiement par espèce, pour sa commande.

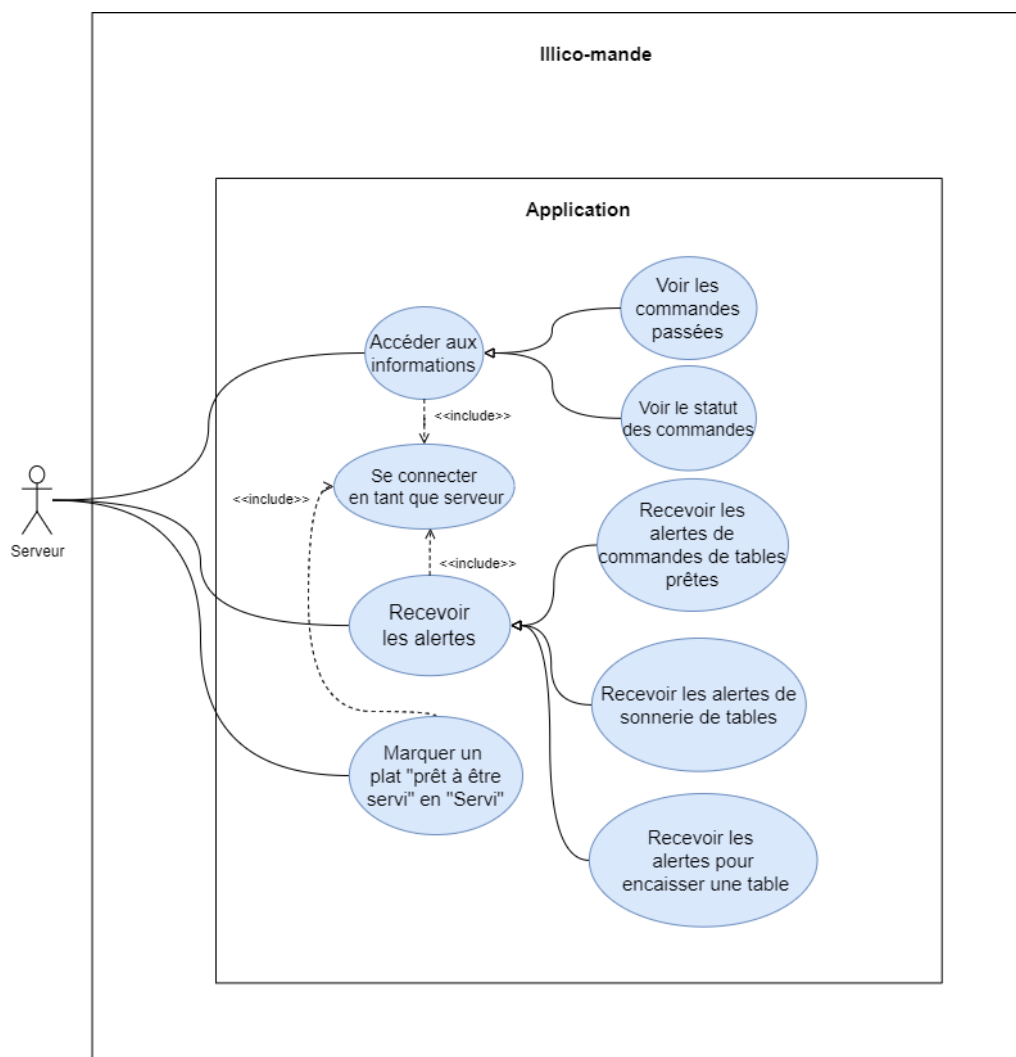


Figure 3 : Diagramme de cas d'utilisation (côté Serveur)

3. Le Cuisinier :

- Se connecter à l'application du personnel en tant que cuisinier : Le cuisinier accède à l'interface dédiée au personnel avec ses identifiants.
- Voir toutes les commandes à préparer : Le cuisinier voit la liste de toutes les commandes en attente de préparation.
- Agir sur une commande : Le cuisinier peut voir le contenu détaillé de chaque commande, ainsi que la marquer comme "en cours de préparation" ou "prête à être servie".
- Modifier son mot de passe : Le cuisinier peut modifier son mot de passe.

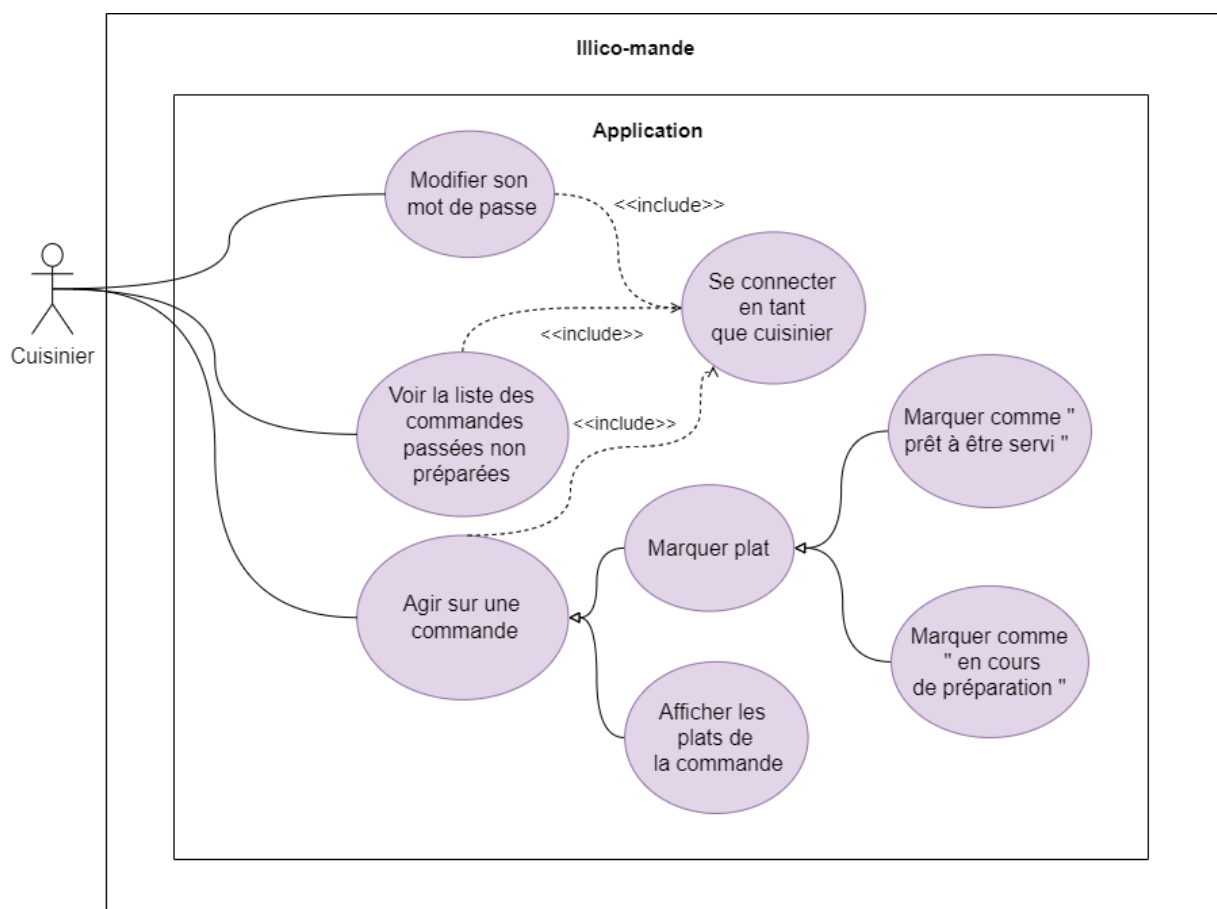


Figure 4 : Diagramme de cas d'utilisation (côté Cuisinier)

4. L'Administrateur :

- Se connecter à l'application du personnel en tant qu'administrateur : L'administrateur accède à l'interface de gestion avec ses identifiants.
- Créer des comptes pour le personnel : L'administrateur peut créer de nouveaux comptes pour les serveurs, cuisiniers ou autres administrateurs.
- Modifier les mots de passe des comptes du personnel : (« inutile de détailler plus »)
- Supprimer des comptes du personnel : L'administrateur peut retirer l'accès d'un membre du personnel au système (administrateur, serveur, cuisinier).
- Modifier les tables assignées aux serveurs : L'administrateur peut changer la répartition des tables entre les serveurs.

N-B : L'administrateur peut faire tout ce qu'un serveur et un cuisinier peuvent faire.

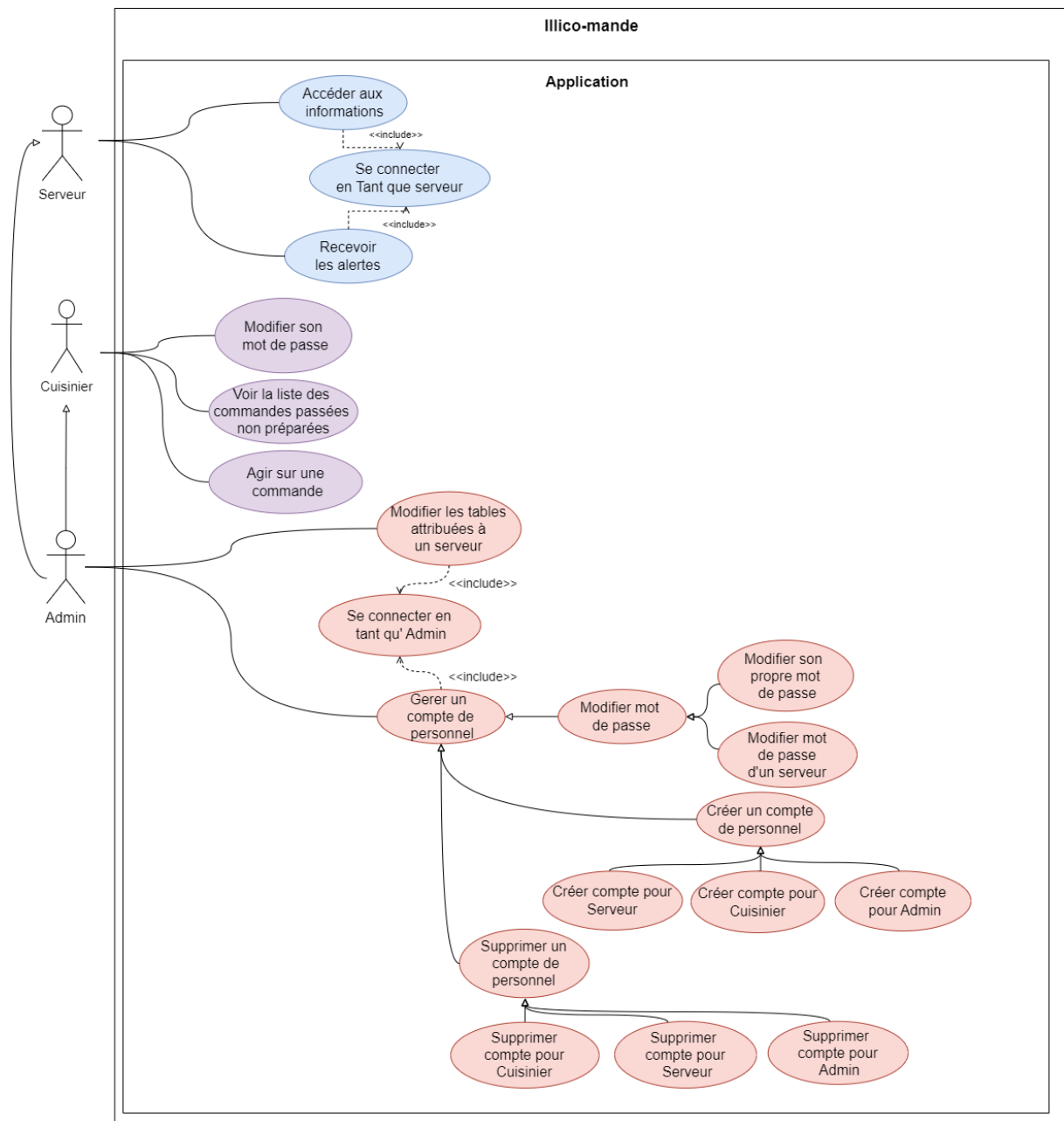


Figure 5 : Diagramme de cas d'utilisation (côté Administrateur)

Pour finir, voici le diagramme de cas d'utilisation (réduit) de notre système dans son intégralité :

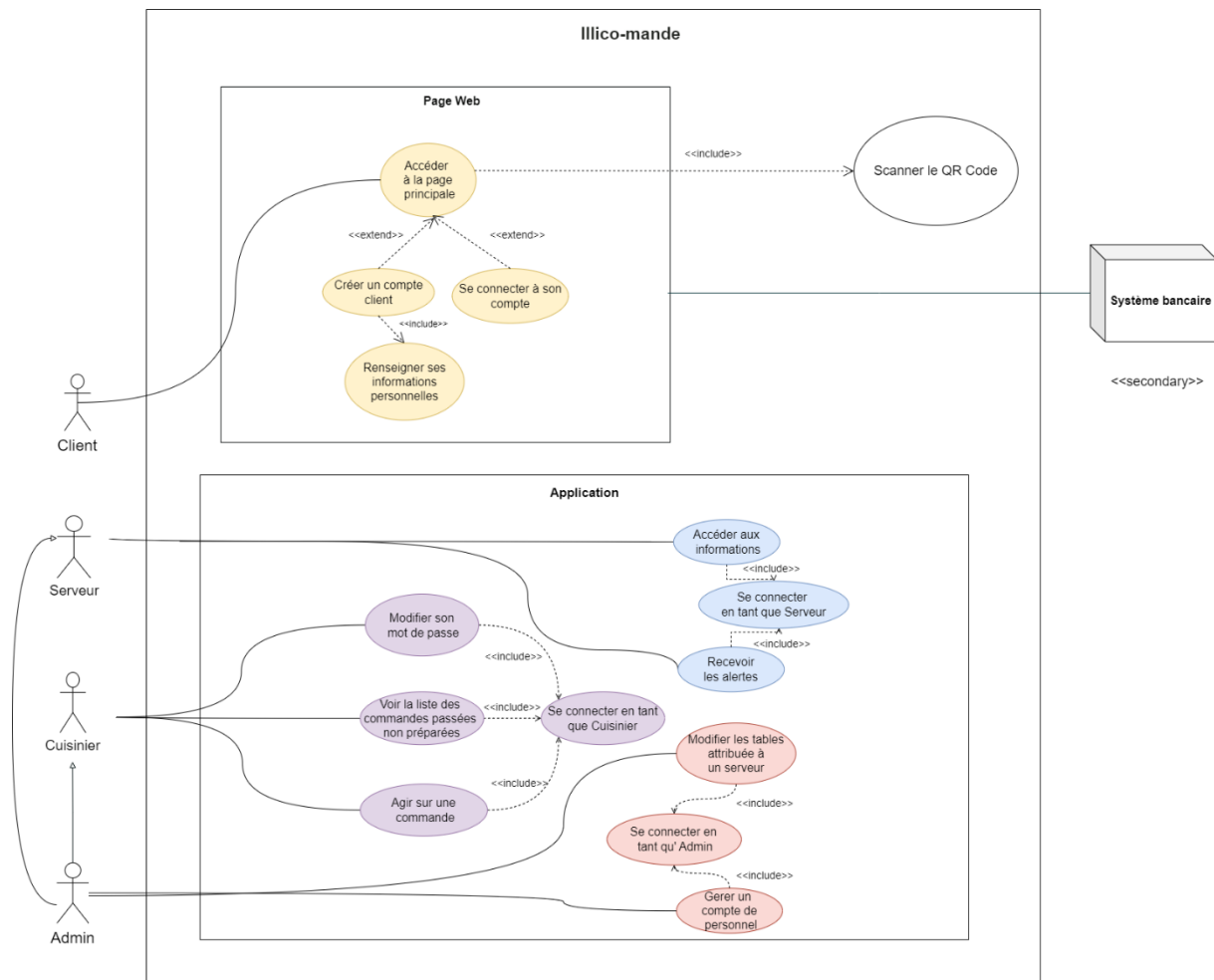


Figure 6 : Diagramme de cas d'utilisation réduit

Nous venons, à présent, de conclure cette partie sur le diagramme de cas d'utilisation de notre système. Passons maintenant à son diagramme de classe.

IV. Diagramme de classe du système

Cette partie de projet permet de passer de l'analyse des besoins des utilisateurs à la conception de la structure interne du système.

Toujours selon la même méthodologie, nous dévoilerons progressivement le diagramme de classe. Nous avons adopté une conception qui minimise autant que possible les dépendances entre les classes afin de maintenir la flexibilité du système et de faciliter les modifications futures.

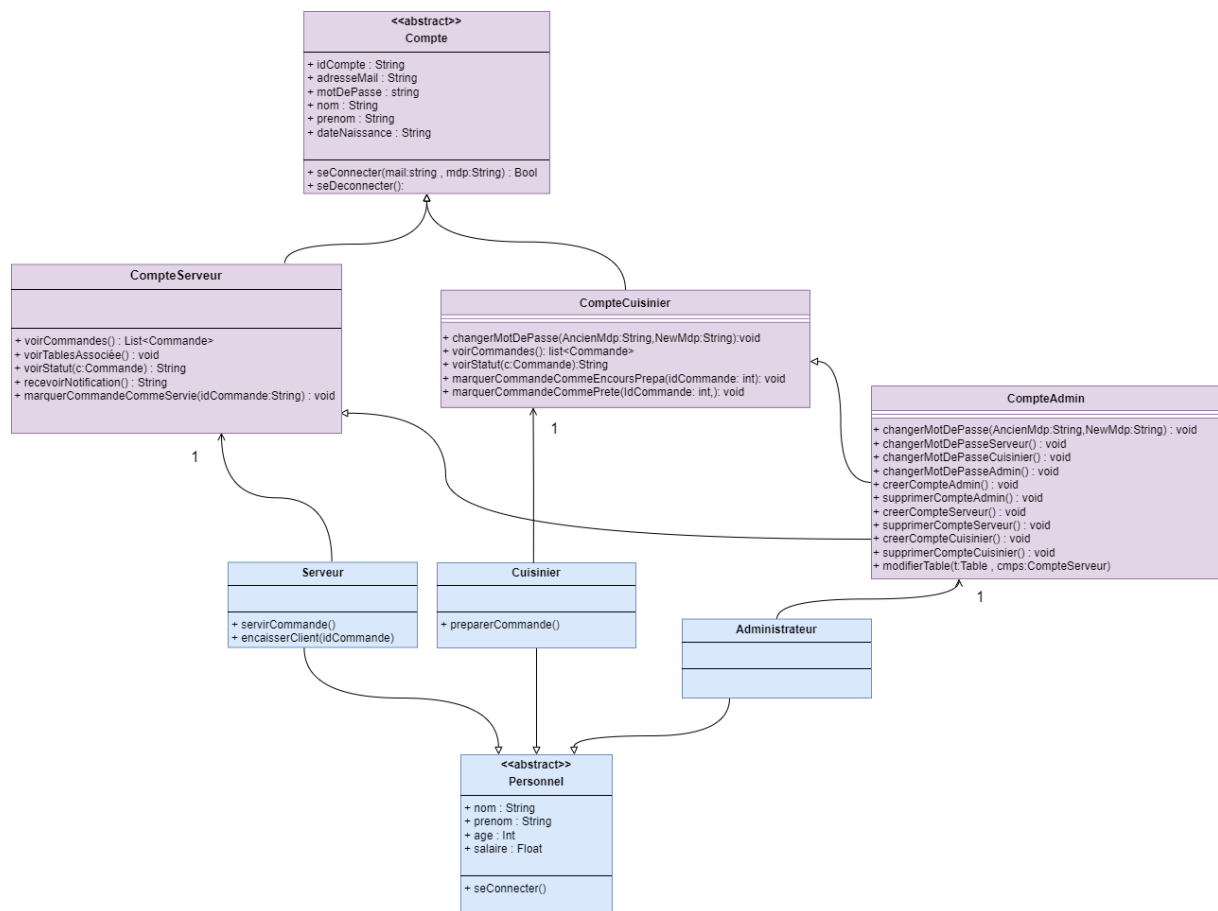


Figure 7 : Diagramme de classe (côté application)

Ce diagramme fournit une conception basique de notre système :

- Classes abstraites et sous-classes :

Les classes Personnel et Compte sont deux bases abstraites. Nous commençons par relier chaque personnel (Serveur, Cuisinier, Admin) à un compte d'application. Le personnel peut être enregistré dans une base de données dédiée ou bien considéré comme entité physique qui agit directement avec le système (Pour mieux comprendre). Le compte peut être un compteServeur, un compteAdmin, ou un compteCuisinier, et éventuellement, un compteClient pour les clients connectés.

- Héritage :

La classe CompteAdmin hérite des deux classes CompteServeur et CompteCuisinier. Cela signifie qu'à travers un compte administrateur, on peut effectuer toutes les actions réalisables via un compte serveur et un compte cuisinier. En plus des méthodes héritées de ces classes parent, CompteAdmin inclut des méthodes spécifiques. Par exemple, une méthode pour changer son propre mot de passe, et une autre pour changer le mot de passe d'un autre administrateur (utile en cas d'oubli). Si un seul administrateur existe, il peut également créer de nouveaux comptes administrateur, ainsi que des comptes serveur et cuisinier.

- Associations et restriction de navigabilité :

Notre structure est conçue pour limiter la navigabilité entre les classes autant que possible, afin d'assurer la simplicité et la sécurité du système. Chaque membre du personnel a accès à un seul compte selon son type approprié.

- Harmoniser et orchestrer les méthodes de classes :

On a pensé à notre système d'une manière à ce que chaque classe reste concentrée sur sa propre responsabilité, tout en permettant une interaction fluide entre les différentes classes. Cela améliore la facilité des futures extensions ou modifications du système. Par exemple, le Cuisinier est responsable de la préparation physique des commandes et peut se connecter à son compte, tandis que CompteCuisinier gère les aspects administratifs et le suivi des commandes, y compris le changement des statuts des commandes.

Après avoir vu le diagramme de classe du côté de l'application (Serveur, Cuisinier et Administrateur), passons à une autre partie de notre diagramme de classes, qui montre cette fois-ci les relations entre les classes du côté Application et les classes Commande et Table.

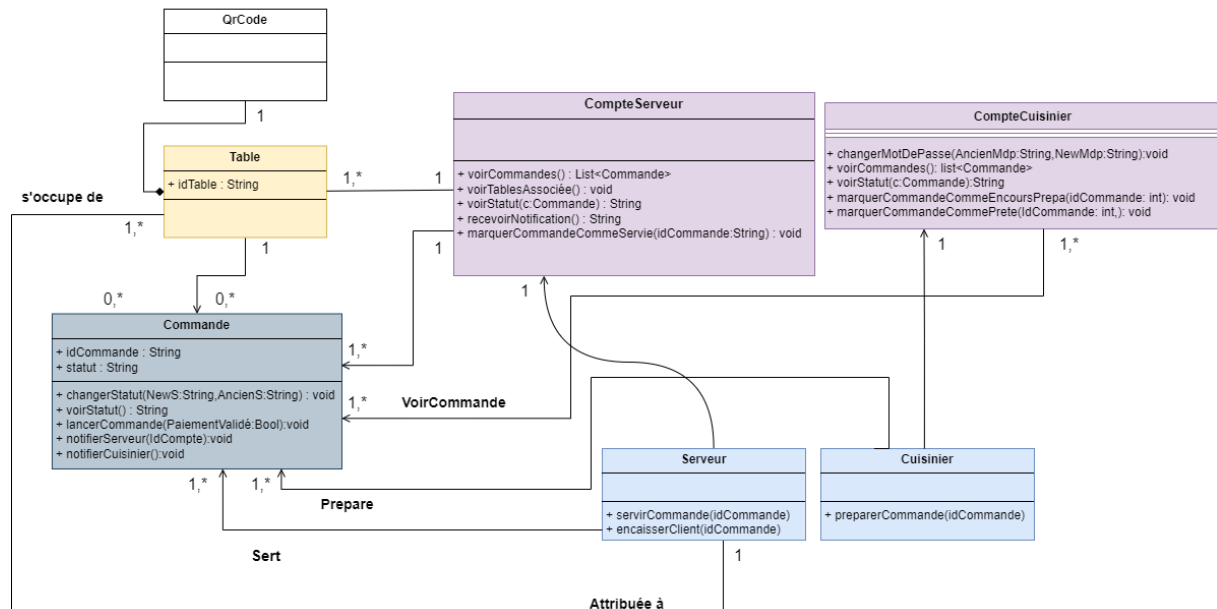


Figure 8 : Lien entre Table, Commande et Serveur, Cuisinier

- Table : Une table est composée d'un QR code associé. Si la table est détruite, le QR code l'est également. À noter qu'une table est associée à un seul serveur qui en est responsable, et un serveur peut s'occuper de plusieurs tables. Une Table peut être associée à plusieurs commandes (la cardinalité des commandes représente le nombre de commandes actuelles et non l'ensemble des commandes qui ont été passées dans l'histoire de cette table (historique)).
- Commande : Une commande est un Panier de Formules et de Plats (que nous détaillerons ultérieurement) qui est validé et payé par la session du client. Autrement dit, lorsque le panier est validé et payé, une commande s'instancie pour être traitée par les comptes des personnels.

La commande est associée à une table, à un serveur qui la servira une fois prête, et à un cuisinier qui la préparera.

Remarque : Notre système considère le client comme une session de navigateur, représentée par la classe `SessionClient`, que nous verrons plus tard. Cela découle du fait que dans les applications web, on gère des sessions utilisateur plutôt que des utilisateurs directement connectés en permanence.

Passons à une autre partie du diagramme de classes.

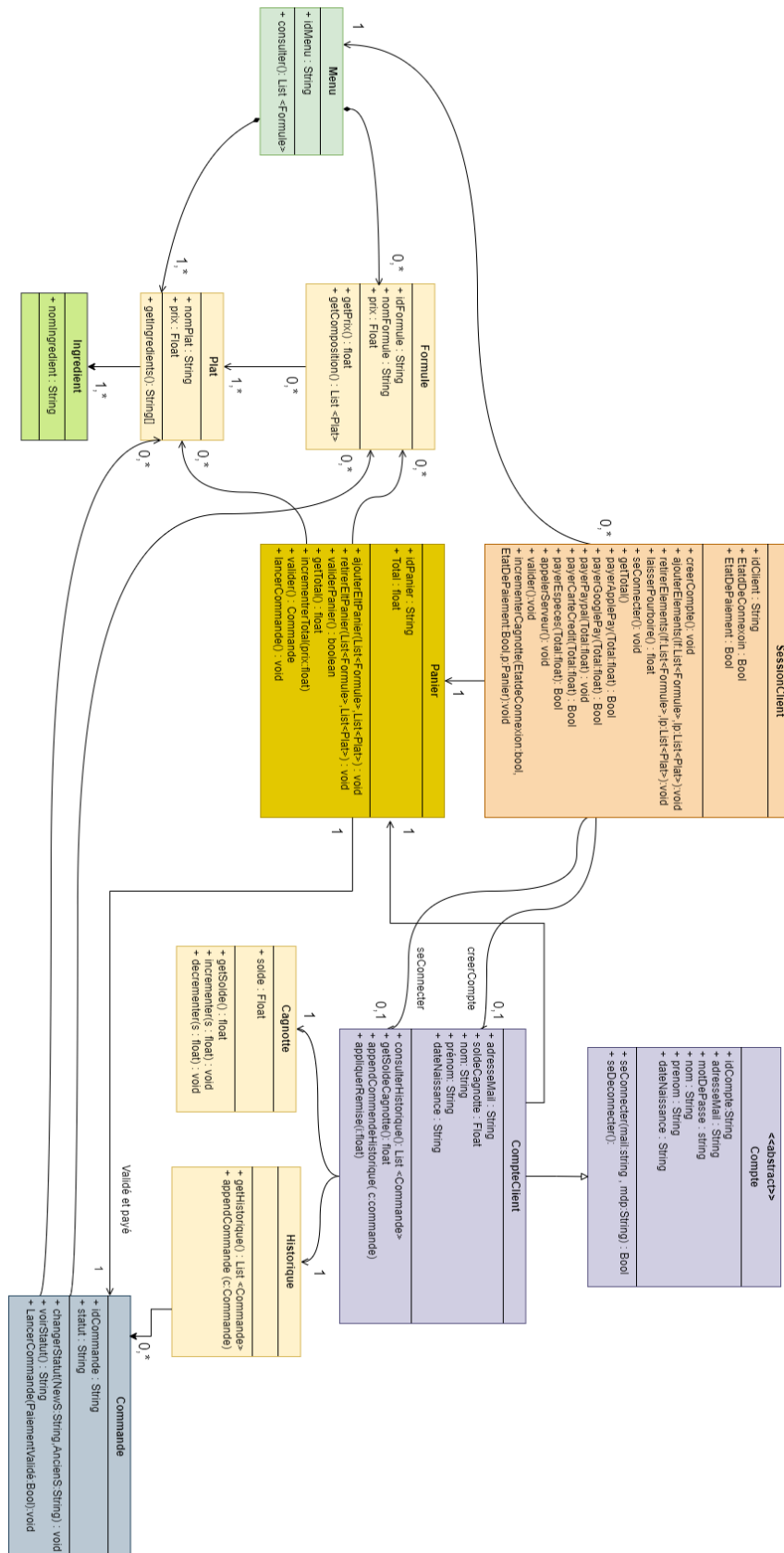


Figure 9 : Interactions que peut avoir le Client avec Menu, Panier et Commande

Cette partie du diagramme s'intéresse aux différentes interactions que peut avoir le Client avec Menu, Panier et Commande.

- SessionClient :

Une fois que le client scanne le QR code avec son téléphone, il est amené à une page web où il est identifié par une session. À partir de cette session, il peut profiter des différents services : voir le menu détaillé, commander, appeler le serveur, et payer.

- Panier :

Pour commander, le client dispose d'un panier virtuel sur la page web, où il peut ajouter les formules et les plats qu'il souhaite. Une fois satisfait, il peut valider le panier et payer, transformant ainsi le panier en commande qui sera visible par le cuisinier et le serveur dans leurs comptes respectifs.

- Menu :

Le menu est composé de Formules et de plats. Les formules sont des combinaisons de Plats, et les plats sont composés d'Ingrédients

- CompteClient :

Le client peut créer un compte ou se connecter (afin d'associer la session à un compte). Cela lui permet de conserver un historique de ses commandes et, dans une future conception, d'utiliser les cookies pour anticiper ses préférences en cas de nouveautés dans le menu. Le client peut également bénéficier d'une cagnotte où il accumule 5 centimes pour chaque euro dépensé, qu'il pourra utiliser comme remise.

- Commande :

La commande est associée aux plats et aux formules.

Dans cette conception, nous avons maintenu la cohérence des méthodes entre les classes pour assurer un flux d'activités qui fait sens. Chaque classe dispose de ses méthodes appropriées pour préserver une certaine indépendance entre les classes. Par exemple, dans la classe SessionClient, il y a la méthode ajouterElement qui prend en argument une liste de formules et de plats pour les ajouter au Panier. Lorsqu'on passe au Panier, on le valide et l'attribut PanierValide devient vrai. Lorsque le paiement est effectué, l'attribut PanierPayé devient vrai. Dans ce cas, l'appel automatique à la méthode LancerCommande va vérifier la véracité des deux attributs nécessaires avant de lancer la commande.

Nous avons décidé d'adopter une conception où, lorsque la commande est instanciée, elle envoie une notification au serveur associé à la table qui est elle-même associée à la commande. Montrons ce que donne le diagramme de classes complet.

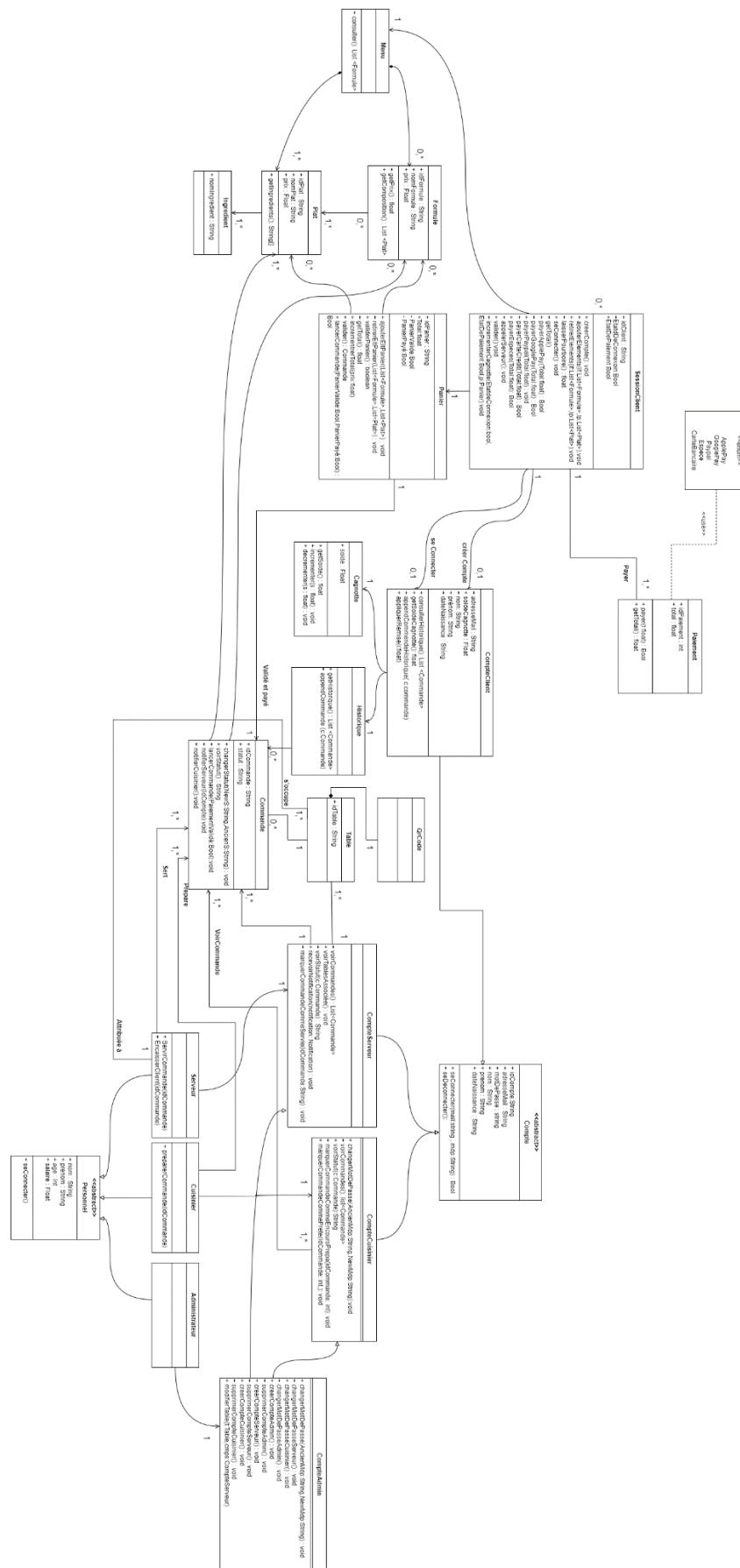


Figure 10 : Diagramme de classes complet

- Paielement :

Pour le paiement, nous avons une classe Paiement qui utilise une énumération pour déterminer le type de paiement (ApplePay, GooglePay, PayPal, Carte de crédit, En espèce). Le comportement attendu du système est que la SessionClient appelle automatiquement la méthode appelServeur() lorsque le paiement en espèce est sélectionné, afin qu'un serveur vienne encaisser le client.

Remarque : Pour ces deux derniers diagrammes (Figure 9 et 10), nous avons été contraints de les disposer à la verticale afin d'améliorer la clarté et netteté de l'image.

Notre diagramme de classes pour "illico-mande" donne une bonne vue d'ensemble du système. On y voit comment les différentes parties (clients, commandes, produits, personnel) s'imbriquent. Il illustre clairement les rôles et interactions de chaque élément du système.

A présent, nous allons effectuer le diagramme d'états-transition de la classe Commande.

V. Diagramme d'états-transition de la classe « commande »

Afin de réaliser un diagramme d'états-transition, il faut avant toute chose, identifier les différents états, ainsi que les transitions entre ces états.

Faisons cela pour notre chère Commande :

Avant d'être passée, une commande est initiée. C'est le cas où le panier n'est pas encore validé et payé.

Lorsque ces deux dernières actions (valider et payer) sont effectuées, la commande passe au statut « Commande créée ». Cela signifie qu'elle est prête à être prise en charge par la cuisine. Lorsqu'un cuisinier commence à travailler sur la commande, elle entre dans l'état "En cours de préparation". Une fois tous les plats préparés, le cuisinier marque la commande comme "Prête à être servie". Enfin, lorsqu'un serveur apporte la commande au client, il la marque comme "Servie", son état final.

Remarque 1 : Le premier point de décision est utile, car nous avons jugé qu'il est possible qu'une commande soit déjà prête, et qu'elle n'ait donc pas besoin d'être préparée.

Remarque 2 : Le deuxième point de décision a été pensé de telle sorte à pouvoir faire évoluer notre architecture. Voici, ci-dessous des exemples de ces améliorations possibles.

1. Intégration d'un compteur de temps :

- Objectif : Suivre le temps de préparation de chaque plat.
- Action : Notification au cuisinier après un délai spécifique (ex : 5 minutes).

2. Système de rappels :

- Objectif : Assister les cuisiniers dans la gestion du temps de préparation.
- Exemple : Rappel automatique après 20 minutes pour un plat au four. (Cela pourrait s'écrire comme tel : `entry / after(20min) : notifierCuisinier()`)

3. Gestion des imprévus :

- Par exemple, dans le cas d'un plat brûlé ou mal préparé ; le cuisinier peut remettre la commande en état "En cours de préparation" ; ce qui fera réinitialiser un compteur de temps.

Ces améliorations viseraient à optimiser le processus de préparation, réduire les erreurs et améliorer l'efficacité globale du service en cuisine.

Ci-dessous le diagramme d'états-transition d'une Commande.

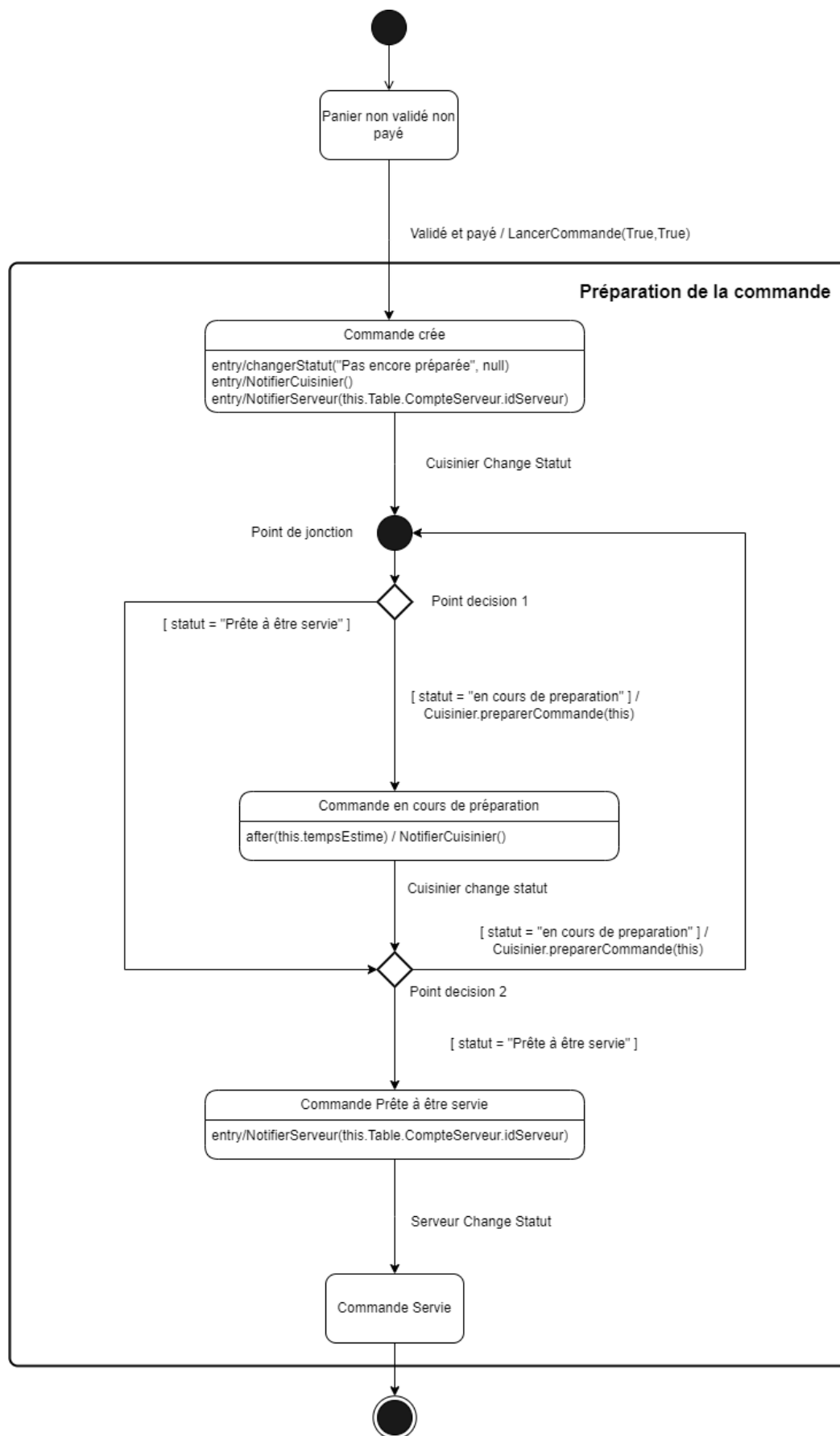


Figure 11 : Diagramme d'états-transition d'une Commande

Pour conclure cette partie ; le diagramme d'états-transition d'une Commande offre une représentation claire du cycle de vie d'une commande dans notre système. Il illustre les différentes étapes, de l'état "Créée" à "Servie", en passant par les phases de préparation.

VI. Conclusion

Pour conclure, ce projet "illico-mande" représente une innovation significative dans le domaine de la restauration, visant à simplifier et automatiser le processus de commande. Notre travail de modélisation UML a permis de structurer et de visualiser les différentes composantes de ce système complexe.

À travers le diagramme de cas d'utilisation, nous avons clarifié les interactions entre les utilisateurs et le système, mettant en évidence les principales fonctionnalités offertes aux clients, serveurs, cuisiniers et administrateurs. Le diagramme de classes a fourni une vue d'ensemble de l'architecture du système, illustrant les relations entre les différentes entités et leurs rôles respectifs. Le diagramme d'états-transitions de la classe Commande a, quant à lui, permis de représenter le cycle de vie d'une commande, de sa création à son service, offrant une compréhension claire du processus de traitement des commandes.

Cette modélisation UML a non seulement servi à structurer le système actuel, mais a également ouvert des perspectives d'évolution, notamment en termes d'optimisation de la gestion des commandes en cuisine.

En résumé, ce projet démontre l'efficacité de la modélisation UML dans la conception de systèmes informatiques complexes, fournissant une base solide pour le développement et l'implémentation future du système "illico-mande".

Nous tenons également à exprimer notre sincère gratitude envers vous pour ce projet enrichissant. Cette opportunité nous a permis d'approfondir nos connaissances en UML et de développer des compétences pratiques essentielles dans la modélisation de systèmes complexes. Ce travail a non seulement renforcé notre compréhension théorique, mais nous a également préparés aux défis réels de la conception logicielle. Merci pour cette expérience formatrice qui contribuera indéniablement à notre croissance professionnelle.